

Company: Dolat Capital

Job Profile: Quant Developer

Offer Type: [Internship + Placement]

Internship Stipend: Not Specified

CTC: 11 (10 fixed, 1 variable)

Location: Mumbai

Process Date: 20/8/2025

Placement Process:

The process was as follows:

Online Assessment	Based on Python, C++, Easy/Medium DSA
Technical Round	C++/Python in depth, Medium DSA, Probability Stats
HR Round	Standard HR questions

This is going to be a long one.

1. **[Round 1 - Online Assessment]** [No of candidates who appeared for this round: 60-70]

The OA consisted of 5 sections and was conducted on Imocha safe assessment browser:

- C++ MCQs
- Python MCQs (mainly NumPy and statistics-based)
- Aptitude MCQs (standard, mostly easy)
- Python coding
- C++ coding

The coding questions were DSA/implementation based. Some candidates received harder problems, but mine were very straightforward.

Python coding question:

You are given transaction data with categories and amounts. The task was to compute the total amount spent per category and print it as a float. Essentially, maintain a dictionary with category -> total amount.

C++ coding question:

Given an integer k and an array, compute the running average over the last k elements.

Example:

```
arr = [100, 200, 300, 400], k = 3
```

```
i = 0: 100 / 1 = 100
```

```
i = 1: (100+200) / 2 = 150
```

```
i = 2: (100+200+300) / 3 = 200
```

```
i = 3: (200+300+400) / 3 = 300
```

I don't remember the mcq questions well, but knowing intermediate c++ and python helped me. Also python mcqs were mostly around statistics and numpy functions like cumulative sum, median, etc.

The OA was in a very unexpected format and since the languages to use were so limited, many people gave up halfway in the test. (Also quite a few for being flagged as cheating)

My OA round, unlike everyone, was very eventful. Initially, I did not have access to a Windows laptop, which was required for the test, so I had given up hopes of even trying for the OA, but fortunately, my friend [Asim](#) helped me at the last moment. The OA program crashed midway (saying that I'm sus) and I had to wait till I got a fresh new link. The HR and support team were really very nice that they helped me and also arranged the new test (I was able to prove my innocence). I gave the OA after everyone finished and they stayed with me for invigilation as well. I hurriedly finished the OA and in the hurry, I wasn't able to think logically and thus, I couldn't solve the c++ dsa question. Again I had no expectations that I would even make it into the interviews.

2. **[Round 2 - Technical]** [No of candidates who appeared for this round: 9]

My panel was conducting 1-1.2 hrs of interviews, mostly focused around c++ (in depth), medium/hard dsa, and probability statistics. My interview was by far the longest i.e. 2 hrs. It started with my introduction, surprisingly they weren't as interested in my projects and internship and only inquired basic information on it while also asking whether they are in c++ or not.

They then asked me if I am confident in c++ or python, choosing c++ he asked me what's the difference between struct and a class. He also asked me what's the difference between c and c++, I answered having objects, to which he responded that we can make objects in c too (he's right about it technically speaking). Then they asked me what I know about oops in c++. I answered with 4 pillars of oops. Then he asked me which one I know the best, I answered polymorphism and explained runtime and compile time polymorphism, with code examples and also how compiler evaluates them (the compile stage and how runtime is evaluated with vptrs and vtable, I didn't go much deep in this to the function pointers but he was satisfied with my explanation). After that he asked me about the second best oops pillar that I know of, I answered with encapsulation and explained intuitively with real life examples and had to write code for the same.

Then he asked me about my favourite topic in c++, to which I answered templates and then I explained how the compiler evaluates templates (in the compile time itself). Then we discussed about how and when `std::vector` resizes, how can we check when the vector resizes. I answered that we can check resizing by seeing the operation time consumed. He said that it may be possible that the CPU is too fast and we can't even measure the difference. Then he answered it for me, saying that the address of the vector changes. Hearing this, it clicked to me so I explained it further that

resizing is done by creating a new vector of double the “capacity” and then the elements are copied into the new vector. He then also asked me about size vs capacity in this context and asked whether I can implement it. After which he asked me the difference between `std::array` and `std::vector`.

The conversation then moved on to memory management, starting with the difference between “malloc” and “new” keywords in c++ (new just calls the destructor of the class automatically). Then I was told to write some code about runtime polymorphism using the malloc keyword, it was basically creating class pointers and showing runtime polymorphism with it. (I was used to using new instead but the interviewer kindly gave me a hint that it needs typecasting when used with malloc). Then I also explained about upcasting and downcasting using Class pointers.

After which he asked me why even bother using heap memory? I answered that we can use it for storing large objects, to which he replied that if I have lots of memory it won't be a problem. Then I said that we can use it for dynamically allocating memory at runtime, when we don't know the size of the object at compile time. To which he confirmed that I am correct and added that heap memory is much faster (I think he meant it is faster for larger objects). I also explained about the need for destructors citing examples of sockets and files.

He asked me if I knew anything else in c++, so I answered with smart pointers, after which he asked me about why they are used. Then he asked me whether I know about mutexes and locks, after hearing a yes, he asked me what is a spin lock? I couldn't answer it. Next, he also had a question which I found tricky because I didn't understand the question well enough. He asked me whether some memory allocated in the constructor for a non member of the class, then when do we release it.. I was assuming he is talking about a stack memory variable, but he meant heap one. The answer was to deallocate it in the constructor itself.

Then he moved on to asking if I know about any other core subjects. Judging from the first part of the interview, I felt that saying any topic which I don't know absolutely 100% would be a disaster. So I answered with OS CPU scheduling and deadlocks. After explaining that, he asked me whether I knew about TCP/UDP (which was written in the JD requirements btw) and which core subject it belonged to. He asked me difference in TCP, UDP and an example where to use them.

Then they came to dsa questions which I honestly didn't do well, first they started with standard min stack question, given a stack, create a getMin with $O(1)$ complexity. I initially forgot the two stack approach code so I gave them a single stack of `pair<int, int>` where `pair.first` is the element and `pair.second` is the minimum till now. He acknowledged the solution was correct but he specified that I cant change the data type of the stack. then I told him about the two stacks approach, to which he replied that it is taking too much space complexity, finally after 15 mins of me struggling he told me the answer that we can instead store pseudo elements in the stack (I saw this approach at neetcode.io/solutions/min-stack but I forgot it during the interview).

Then he gave me a detect cycle in linkedlist, which as well I forgot the iteration condition, and then figured out after some time, communicating entire thoughts continuously is a must. Then he asked me to also find the node starting the cycle while pointing out possible segmentation fault in my code in three places. The last dsa question I was asked was find the missing element. After listening to this I felt very relaxed because it's an easy question. I answered with xor of array method, to which he nodded. Then he asked me about another approach, he said let's say if it's sorted, what

now? I was a bit stumbled so I answered binary search without thinking. He clarified I was wrong. He then said me that it needs to be faster. I assumed he meant faster than $O(n)$, and I went ahead scratching my head for another 5 minutes. I answered that all I can think of is Sum of natural numbers formula. He said it's in the right direction. But I kept on thinking that he wanted $O(\log n)$ or $O(1)$ solution. He clarified this miscommunication and then I wrote the code for it using `std::accumulate` method, to which he asked what's the complexity of that method.

Later the second panelist started by asking if I had done probability and statistics during my course. He started with linear regression, asked me about it, and then gave me some data to perform it and evaluate it.. I said I dont recall the formula. He then asked me about uniform distribution, to which I answered slightly correctly. I explained my thinking with examples. I also explained about PMF and PDFs (in general what they are). After which he looked over my projects, one with ML caught his attention which was "DsaBuddy", he asked for explanation on what I did in it. It was a Random Forst model predicting next dsa problem to give to a person to learn, I told him about features used. He asked me about Random forst and decision trees, I explained them in detail. He asked me how can I get which feature is most important, which I forgot at the time. He also asked me to give an example decision tree based on my project and then he asked me why use random forest over decision tree. Then he also asked me about some Classifier in tensorflow which I couldnt answer. Later, he asked me about Lasso and Ridge and what's difference between them.

He then asked me about a logical puzzle, given a revolver gun which has 6 slots and two bullets placed consecutively. Which is higher probability that next shot will fire the bullet? There were two cases here that in the first fire, the bullet was not in the slot, and second case that bullet was present and it had a malfunction. I plotted down the cases and in each case calculated probability of shuffling and not shuffling. He later informed me of a mistake that after the first misfire, the revolver moves one section ahead automatically which I didnt account.

Then he asked me about gaussian distribution, to which I could explain what it is and the standard deviation and mean in it. he asked me about expectation of x general formula which I couldn't recall. Then he asked me about the gaussian distribution's PDF which I could remember only some parts, and writing just some parts of it he was satisfied with it.

3. **[Round 3 - HR]** [No of candidates who appeared for this round: 9]

Fortunately, every candidate was given the chance to appear for the HR round as soon as their Technical round was done. This meant all 9 candidates appeared for both the interview rounds without any shortlisting.

The HR interview was short, also luckily I met the same HR who stayed with me in the OA. I started with my introduction, then I was asked about my parents' professions and whether I have siblings. I was also asked about 3 positives and 3 negatives about myself (classic strengths and weaknesses. I made a slight mistake that after recognizing this question I kept on saying "strength" and "weakness" instead of positive/negative as they said). Then they asked me why I'm pursuing this degree, whether I have plans for higher studies, how I would rate myself in c++ and python and finally if I have any questions for them. I answered all the questions citing story-type examples from my work experience and school and college life.

After this round, two students, including myself, were selected, while two others were asked to appear for a confirmation round. Since no one was selected in the confirmation round, the company decided to conduct another hiring process, similar to the earlier one, for students who had not attempted the Online Assessment. In this process, one student was selected.

[In total 3 students were offered the job]

Lessons learned:

Maintain your composure at all times. I can't remember how many opportunities I lost due to this. My ability to perform in stressful situations was almost non-existent because for practicing dsa I was always in a learning attitude. In an interview this was never the case since it can determine your future. I would recommend doing codeforces and leetcode contests as much as you can. For this particular interview, I never really had any hopes because I thought my dsa skills are not worthy of a quant position. So I was again in the same learning attitude.

Be 110% confident in yourself, chances are that then they will perceive you at 70%. (But please justify it too) I never made my introduction good to catch attention, I was only speaking about what I did. No one is interested in what you did. You need to communicate what you can do that can benefit them. I learned this from my friend [Asim](#). So in this interview, I played entirely on my strengths, i.e. understanding the language at a very deep implementation level. I had web development projects and experience, but in my intro I told them that I also made a small mini project in rust by parsing my linux configuration file. I use Arch Linux and I use vim btw. (I actually said this in the interview too)

Don't let any incorrect answer derail your confidence. I initially had a very good interview in the first half and mid was very bad, but I didn't mind it much so I caught up in the last parts.

Be humble. Placements are a humbling experience. The earlier you accept it, the easier it gets for you.

Communicate your thoughts well, and remember it is a two way street. My entire DSA answers were very bad, but since my interviewer was very nice, he gave me feedback on whether I am going in the right direction or not. In most of the common cases, you will have to judge this by yourself by observing the interviewer very attentively. I think I never realised this since I was always focused on the right answer and speaking it as fast as I can.

Have a good resume such that it looks filled completely, leaving any empty whitespaces signals that you haven't done anything and memorize your resume on your fingertips so that any keyword or any related keyword is known to you and you can talk about it for a good 2 minutes at minimum. For example, I mentioned "BERT" model in my projects section, then the interviewer asked me about Transformer architecture along with BERT. Also single keyword also has variations to ask on, like on WebSockets I was asked about the http upgrade packet to ws:// protocol and another time I was asked about how sockets communicate (classic socket bind/accept diagram). If you are short on time, I recommend you to look out on at least 25 common interview questions on each tech keyword you mention on your resume.

Practice speaking a lot. I was a dead silent introvert (my literal speaking capacity is 2-3 hrs after which I have a sore throat). I followed [Vinh Gang youtube](#) and [Matt Huang youtube](#) to learn how to communicate better. It does not matter how good you are if the person in front of you doesn't understand what you say.

Fomo is only as long as you let your emotions control you. In my friend group I was one of the last people to get placed, and most of the higher paying companies already came and went. It is easy to say but hard when it's on you. I did feel panicked but I was confident due to having a solid internship experience.

Saying that Placement is luck based is an understatement. It is Swiss cheese luck, there are lots of factors, many of which you can't even think of, which are needed to have good luck in order to succeed. I hope my case is evident for this point. But you do need to do the needful so that when the opportunity strikes you are ready to go in.

Research the company very well before giving any interview. I have been asked if I know the company's business even in technical rounds. I used perplexity deep research for this and it helped me understand their businesses as well as the HR stuff. Highly recommended. Also have a look at the [Companies Expert youtube](#) for prepping on all those HR questions. HR questions sound stupid at first, but they are very psychological in nature. Understanding what questions mean to ask what is very very important.

The last lesson I would say is to **keep persevering**. I was not able to accept the rejections at first, but then I heard my Uncle's story:

He graduated from a Tier-3 college and remained unemployed for 1.5 years in poverty. Eventually, he approached placement agencies that provide training and subsequent job opportunities in exchange for 50% of the candidate's salary after securing employment and also an upfront non refundable fee in lacks. While most candidates did not receive opportunities through this route, he succeeded and even outperformed a candidate from NIT Trichy in the interview. Today, he is working at Intel in Silicon Valley.

He also struggled for admission into college, it was raining heavily in Mumbai the day he went for admissions, and almost the entire college was empty because of the immense rains, he got the admission because he reached there when only 3 students came that day.

There are many such stories that we don't know the struggle behind. I am lucky that my employment was not critical for my family's survival. If that's not you, then I hope God bless you.

Resources I used (ordered in importance I gave them, YMMV):

DSA:

- I initially did some CP on codeforces and Atcoder, solving around 30 Leetcode problems, so before placements I did 90% of Neetcode 150.
- Overall I would say my strategy was bad, I was ok at Graphs, Dp but I wasn't as good at problem solving so I couldn't clear elite company OAs. Not knowing standard leetcode problems also put me at risk of bad interviews.
- I would suggest doing CP for at least 6 months, regardless of your interest in it and memorizing NC 150, in the sense that you can recognize the type of problem from NC 150 list. This would give the best of both worlds but I was a bit late in doing so.
- I wanted to do the striver sheet earlier but since it had 179 problems I couldn't have spared time for it.
- So for learning, I watch Striver A-Z course YouTube playlists on each topic like dp, graph, stacks and queues, etc. And for practicing I used Neetcode. (highly recommended)

- I was particularly bad at managing stress during OA and interviews, many times I went blank only to come out and solve the entire solution. Solving CF or LC contests can help with this.
- I had started CF with CP 31 sheet: <https://www.tle-eliminators.com/cp-sheet>
- I also solved a searching and sorting, dp, sliding window from CSES problemset to learn some patterns/ways to do things: <https://cses.fi/problemset/>

Aptitude:

- Since I was a DSE, I lacked good math skills, so I used this youtube playlist to gain basic understanding: [Aptitude Preparation for Campus Placement - YouTube](#)
- Then I went on this playlist but only for very important topics like speed-dist, trains, mixtures, etc: [Quantitative Aptitude Tutorials - YouTube](#)
- Then after that I felt I lacked speed so I kept on watching random trick videos on "imran sir maths" yt channel. By far this helped me most. I would suggest his videos as a must watch for "Time & work" and "Coding Decoding" topics. I was able to solve those questions in 30-45 seconds after some practice.
- The standard indiabix is also good, I personally never did it for all topics because many questions were too repetitive, and it takes a bit longer.

SQL:

- Since I had good fundamentals understanding, this wasn't even an issue. I was able to solve many Hard SQL problems just by intuition, but sadly those didn't accept any partial answer like Dsa questions.
- If you have no idea on sql syntax, start with this video and go in depth on joins, etc. I can vouch for this since I learned mysql from here about 4 years ago in my diploma: [SQL Tutorial - Full Database Course for Beginners](#)
- After knowing this, if you have time, please do solve [Leetcode Sql 50 sheet](#). And try to go from naive to optimal query by learning as many new functions as you can.
- Once done with this, go onto understanding case statements and window functions in sql. I will suggest only 5 most common window functions are more than enough. I learned these from chatgpt. Learning case statements is best ROI on your time vs frequency of being asked.
- Learn about unions, I personally never used them but it is a good to know for your tech interview.
- After this, you can learn about Date and String functions. Understand how to use them and also memorize all the formats like %y, %B, etc. Learn about 20 most commonly used functions in both categories.
- SQL OA questions 90% of the time revolved around manipulating the type, extraction in string and dates like using instr(), left(), trim(), date_format(), etc. Know well about this.
- Finally boss of all topics, recursive CTE. I never learned this topic, but I forgot the count of how many OAs I couldn't solve just because I didn't know this.

HR questions:

- I only ever gave one HR interview, so I am not the best person for advice on this. I read just this one article with 50 hr questions: <https://www.interviewbit.com/hr-interview-questions/>
- I also had memorized 5 incidents, like 5 positive ones and 5 negative ones. Negative ones need to show your learning and how you overcame it, not that u still have it.
- I used one of these incidents in my "strengths" answer, where I said I have high versatility, citing example of my work at navy. I won't tell that to you here :)
- Also have at least 60-70% understanding on what company does and what they are. I used perplexity pro a lot to research about companies in one go.

Tech mcq stuff:

- I never really had to prepare for it because I was a core dev profile. I would say know about these topics well:
- Git: it was asked in workindia oa in detail, about commands.
- Java/Oops: many OAs asked java code for oops and their output.
- Python: 2-3 OAs also asked about python, won't say super necessary, but knowing more has always been helpful for me.
- System design: I heard many places asking about nosql, horizontal scaling and all, I will say Gaurav Sen's youtube playlist on this is more than enough.
- General languages: I recommend roadmap.sh for understanding depth on tech topics, it is an overkill though, so for short and sweet u can see: <https://learnxinyminutes.com/>
- Also keep a deep understanding on every tech keyword you mention on your resume. I was asked about WebSockets, the flow of info, the starting http upgrade msg etc.

OOPS:

- I had a solid understanding on oops because of my diploma. I will say these topics are important:
- 4 Pillars of OOPS all with a real world example.
- Java inner classes, static inner classes and stuff. (vrimp)
- Constructors logic, when which one gets called on which object.
- Know about the final keyword on all classes, methods and variables.
- Know all ways to use super and this keyword.
- Know access modifiers.
- Know about Object class methods like hashCode(), toString(), etc.
- Know about deep and shallow copying (in all languages preferably, Python, Java and C++)
- instanceof operator
- Why and how string is immutable in java
- Composition vs Inheritance (vimp for interviews)
- I won't say this list is exhaustive, so please refer JavaTpoint for OOPs in Java. best resource for it, but takes time.

Schema Design:

- I won't say this is important but may get asked sometimes. This is just writing down relations and/or making ER diagrams.
- Important to know about:
- ER diagrams, EER constructs.
- Plot out entities and just keep on joining them with relations.
- Be alert when making relationships and entity structure, you may get told to write a query for the same.

Finally, if you wish to connect or ask any questions, feel free to reach out to me on my [Linkedin](#).

Wish you the best of luck.